

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 03 -

GERENCIAMENTO DE TRÁFEGO NO KUBERNETES

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	18
O QUE VOCÊ VIU NESTA AULA?	20
REFERÊNCIAS.....	21

EMANIP

O QUE VEM POR AÍ?

Nesta terceira aula, vamos além da configuração básica do Kubernetes e mergulharemos em como o tráfego realmente é gerenciado dentro de clusters modernos. Você descobrirá como o Nginx Ingress Controller atua como a porta de entrada inteligente das aplicações, aplicando roteamento por host e path para organizar e expor diferentes serviços de forma centralizada.

Mais do que apenas roteamento, veremos como proteger esse tráfego com TLS e cert-manager, automatizando a emissão de certificados e aplicando boas práticas de segurança que são indispensáveis em ambientes de produção. Ao final, você não apenas entenderá como publicar aplicações com Ingress, mas também como garantir disponibilidade e segurança, habilidades essenciais para qualquer engenheiro(a) DevOps e arquiteto(a) de soluções.

HANDS ON

Nesta aula, você será conduzido(a) a uma prática fundamental no gerenciamento de tráfego em Kubernetes. O primeiro passo será instalar e configurar o Ngin x Ingress Controller, responsável por interpretar as regras de Ingress e atuar como ponto de entrada único para o cluster. Em seguida, criaremos serviços de back-end e validaremos o acesso inicial via HTTP, estabelecendo a base para o roteamento avançado.

Na segunda etapa, exploraremos o roteamento por host e path, criando manifestos de Ingress capazes de direcionar requisições para múltiplos serviços. Você aprenderá como organizar suas aplicações sob diferentes domínios e caminhos, além de aplicar a reescrita de URL, garantindo que o back-end receba o tráfego no formato esperado. Essa prática mostrará como o Ingress oferece flexibilidade e simplificação em arquiteturas de microserviços.

Por fim, avançaremos para a camada de segurança, instalando o cert-manager, configurando um ClusterIssuer para a emissão de certificados com o Let'sEncrypt e habilitando TLS nos Ingress Resources. Validaremos o acesso via HTTPS e discutiremos as boas práticas de segurança em produção. O objetivo é que você não apenas domine o roteamento no Kubernetes, mas também saiba proteger suas aplicações com certificados válidos, automatizando todo o ciclo de vida do TLS.

Exemplo de aplicação de Ingress com TLS:

```
kubectl apply -f ingress-tls-app.yaml
```

Código-fonte 1 – YAML Exemplo de aplicação de Ingress com TLS no Kubernetes
Fonte: Elaborado pelo autor (2025)

O código-fonte e os exemplos práticos de cada exercício estão organizados no [repositório](#) do curso.

SAIBA MAIS

A jornada pelo gerenciamento de tráfego no Kubernetes nos levou por um caminho fascinante, desde os fundamentos da orquestração até a blindagem das nossas aplicações com as melhores práticas de segurança. Agora, é o momento de aprofundar cada um desses pilares, explorando os detalhes técnicos, as nuances e o "porquê" por trás de cada decisão. Prepare-se para solidificar seu entendimento e dominar a arte de orquestrar o fluxo de dados em um ambiente de produção Kubernetes.

Entendendo o Ecossistema: Containers e Kubernetes em Detalhes

Começamos nossa conversa reafirmando a importância dos **Containers** como a unidade fundamental da orquestração moderna. Eles são, em essência, pacotes autossuficientes que encapsulam tudo o que uma aplicação precisa para rodar – código, bibliotecas, configurações e dependências – garantindo consistência em qualquer ambiente. Essa portabilidade e isolamento são as bases para a agilidade e escalabilidade que o Kubernetes oferece.

O **Kubernetes**, por sua vez, emerge como o "sistema operacional do data center", uma plataforma robusta e de código aberto projetada para automatizar a implantação, escalabilidade e gerenciamento de aplicações containerizadas. Sua arquitetura é composta por dois tipos principais de nós:

Control Plane (Mestre): é o "cérebro" do cluster. Ele toma as decisões globais, como onde alocar um container, monitorar sua saúde e reagir a mudanças. Seus componentes chave incluem:

- **kube-apiserver:** o ponto de entrada principal para a comunicação com o cluster. Todas as interações (sejam de usuários, ferramentas ou outros componentes) passam por ele;
- **etcd:** um armazenamento de chave-valor distribuído e altamente disponível, que armazena o estado de configuração do cluster, os dados e metadados;

- **kube-scheduler:** responsável por alocar os Pods (menor unidade computacional do Kubernetes) aos nós de trabalho, baseado em requisitos de recursos e políticas;
- **kube-controller-manager:** executa vários controladores que gerenciam o estado do cluster, como garantir o número correto de réplicas de uma aplicação.

Worker Nodes (Nós de Trabalho): são as máquinas (físicas ou virtuais) onde suas aplicações realmente rodam. Cada Worker Node possui:

- **kubelet:** um agente que roda em cada nó, garantindo que os containers nos Pods estejam rodando e saudáveis, e se comunica com o Control Plane;
- **kube-proxy:** um proxy de rede que gerencia regras de rede nos nós, permitindo a comunicação entre os Pods e do mundo externo para os Pods;
- **Container Runtime:** o software responsável por executar os containers (ex: containerd, CRI-O, Docker).

A magia do Kubernetes reside na sua capacidade de abstrair a complexidade da infraestrutura. Ao invés de se preocupar com servidores individuais, você interage com recursos lógicos como **Pods** (um ou mais containers compartilhando recursos), **Deployments** (gerenciam a criação e atualização de Pods) e **Services** (expõem um conjunto de Pods com um IP e DNS estáveis). No entanto, para que suas aplicações sejam acessíveis do "mundo exterior", precisamos de uma camada inteligente, e é aí que o Ingress entra.

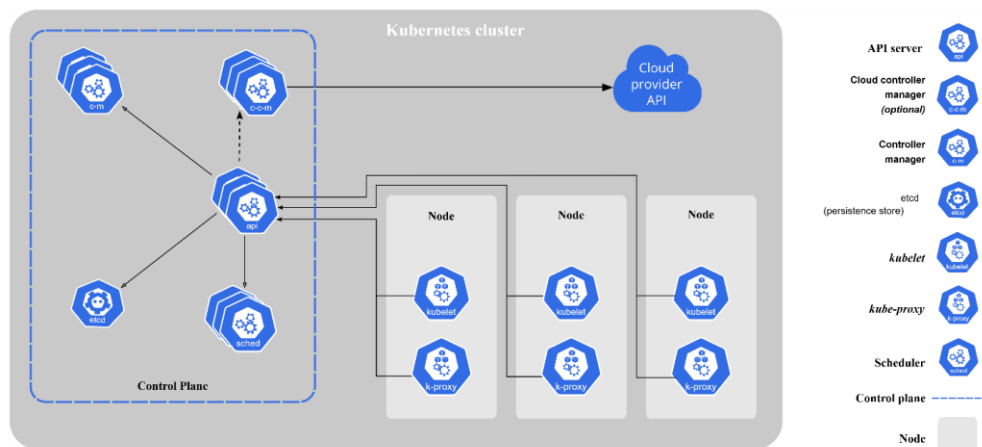


Figura 1 - Componentes do Kubernetes
Fonte: <https://kubernetes.io/pt-br/docs/concepts/overview/components/> (s.d)

O Ingress no Kubernetes: A Solução para Acesso Externo

A exposição de aplicações em Kubernetes pode ser um desafio. Os Services do tipo ClusterIP são excelentes para comunicação interna, mas não são acessíveis de fora do cluster. Já os NodePort expõem serviços em uma porta alta em cada nó, o que se torna impraticável para múltiplos serviços em produção. O LoadBalancer, então, cria um balanceador de carga dedicado em provedores de nuvem para cada serviço, o que pode ser caro e gerar muitos IPs públicos.

O **Ingress** surge como a solução elegante para este problema. Ele não é um balanceador de carga em si, mas sim um **recurso API** do Kubernetes que declara regras para rotear o tráfego HTTP/HTTPS de forma centralizada. Pense nele como um "smart gateway" ou "proxy reverso configurável" na borda do seu cluster.

O Ingress por si só não faz nada; ele precisa de um **Ingress Controller** para funcionar. O Controller é o software que observa esses objetos Ingress, lê suas regras e configura um proxy reverso real, como Nginx ou Traefik, para implementar o roteamento declarado. Isso significa que você tem um único ponto de entrada para o seu cluster, e o Ingress Controller, com base nas regras que você define, direciona o tráfego para os Services internos corretos.

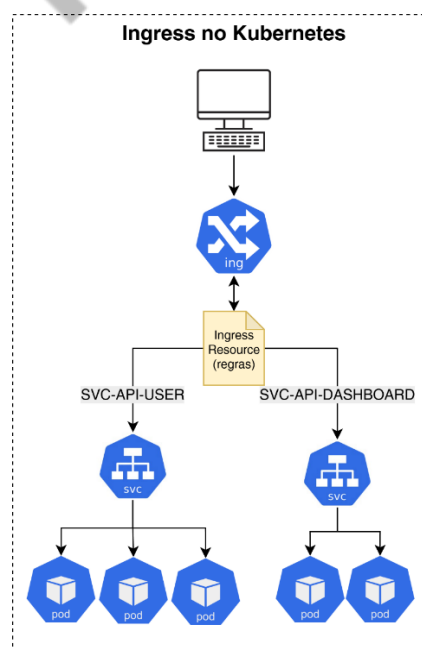


Figura 2 - Fluxo de tráfego detalhado com Ingress Controller
Fonte: Elaborado pelo autor (2025)

Benefícios Essenciais do Ingress:

- **Ponto de Entrada Único:** um único IP público (geralmente do LoadBalancer que expõe o Ingress Controller) para todas as suas aplicações HTTP/HTTPS. Isso simplifica a gestão e reduz custos;
- **Roteamento Flexível:** capacidade de rotear tráfego baseado em nome de domínio (Host) e/ou caminho da URL (Path), além de outras propriedades HTTP;
- **Terminamento TLS/SSL Centralizado:** gerencia certificados SSL/TLS em um único lugar (o Ingress Controller), simplificando a segurança e o desempenho;
- **Reescrita de URL:** permite ajustar o caminho da URL antes que a requisição chegue ao back-end.

NginxIngressController: Configuração e Customização

O **NginxIngressController** é a implementação mais popular e robusta do Ingress, amplamente adotada na comunidade Kubernetes. Ele empacota um servidor Nginx otimizado para atuar como o proxy reverso do seu cluster.

Como ele é implantado? A maneira mais recomendada é por meio do **Helm Chart** oficial. O Helm é um gerenciador de pacotes para Kubernetes que simplifica a implantação de aplicações complexas, agrupando todos os manifestos YAML necessários (Deployment, Service, ConfigMap, RBAC) em um único pacote.

Customização do NginxIngressController:

A flexibilidade do NginxIngressController reside na sua capacidade de ser configurado tanto globalmente quanto de forma granular para cada IngressResource:

ConfigMap Global (ingress-nginx-controller): este ConfigMap armazena configurações que afetam *todos* os IngressResources gerenciados por aquele Controller. É ideal para otimizações de performance, limites globais, ou ajustes de logging que se aplicam a todas as requisições que passam por ele.

Exemplos incluem:

- `proxy-body-size`: tamanho máximo do corpo da requisição;
- `log-format-upstream`: formato de logging personalizado;
- `server-tokens`: exibir ou não a versão do Nginx nos cabeçalhos.

```
# Exemplo de ConfigMap para NginxIngressController (apenas
trecho)
kind: ConfigMap
apiVersion: v1
metadata:
  name: ingress-nginx-controller
  namespace: ingress-nginx
data:
  allow-snippet-annotations: "true"
  proxy-body-size: "20m"
  server-tokens: "false"
# ... outras configurações globais
```

Código fonte 1 - Exemplo de ConfigMap para NginxIngressController
Fonte: Elaborado pelo autor (2025)

Anotações em IngressResources: essa é a forma mais comum e poderosa de customização. Anotações são metadados adicionados ao seu manifesto Ingress que são lidos pelo IngressController para aplicar configurações *específicas* àquele conjunto de regras. Elas seguem o padrão `nginx.ingress.kubernetes.io/<nome-da-anotação>`: `<valor>`. Anotações podem, inclusive, sobrescrever configurações globais definidas no ConfigMap para um Ingress específico.

Roteamento Avançado: Controlando o Fluxo do Tráfego

A verdadeira força do Ingress reside na sua capacidade de rotear o tráfego de forma inteligente, usando as informações do cabeçalho HTTP da requisição.

Roteamento por Host: permite direcionar o tráfego para diferentes Services com base no nome do domínio (hostname). Por exemplo, `api.minhaempresa.com` pode ser roteado para o Service da sua API, enquanto `blog.minhaempresa.com` vai para o Service do seu blog. O IngressController inspeciona o cabeçalho Host da requisição e, se houver uma regra correspondente, encaminha o tráfego. Também é

possível usar wildcards, como *.meudominio.com, para rotear todos os subdomínios para um mesmo back-end, ou para um conjunto de regras que avaliam o caminho.

```
# Exemplo de Ingress para roteamento por Host
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: host-routing-example
spec:
  rules:
  - host: "app.example.com"
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: app-frontend-service
            port:
              number: 80
  - host: "api.example.com"
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 80
```

Código fonte 2 - Exemplo de Ingress para roteamento por Host
Fonte: Elaborado pelo autor (2025)

Roteamento por Path: permite que diferentes caminhos (paths) sob o *mesmo domínio* apontem para diferentes Services. Por exemplo, *minhaempresa.com/frontend* pode ir para um Service e *minhaempresa.com/backend*

para outro. A propriedade `pathType` é crucial para definir como o path é correspondido:

- **Prefix:** corresponde a um prefixo de URL. Se você tem `/api`, ele corresponderá a `/api`, `/api/users`, `/api/v2/products`, etc. É o mais usado.
- **Exact:** corresponde exatamente ao caminho. `/api/users` só corresponderá a `/api/users`, e não a `/api/users/profile`;
- **ImplementationSpecific:** o comportamento pode variar entre Ingress Controllers. No Nginx Ingress Controller, se o `pathType` for omitido ou definido como `ImplementationSpecific`, ele se comporta como `Prefix`.

```
# Exemplo de Ingress para roteamento por Path
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: path-routing-example
spec:
  rules:
  - host: "singleapp.example.com"
    http:
      paths:
      - path: /frontend
        pathType: Prefix
        backend:
          service:
            name: web-frontend-service
            port:
              number: 80
      - path: /api/v1
        pathType: Prefix
        backend:
          service:
            name: api-v1-service
            port:
              number: 80
      - path: /healthz
```

```
pathType: Exact
  backend:
    service:
      name: health-check-service
port:
number: 8080
```

Código fonte 3: Exemplo de Ingress para roteamento por Path

Fonte: Elaborado pelo autor (2025)

- **Combinação de Regras (Host e Path):** a grande flexibilidade do Ingress reside na capacidade de combinar hosts e paths em uma única regra ou em múltiplas regras. Isso permite criar "árvores" de roteamento complexas, direcionando o tráfego com precisão para arquiteturas de microsserviços. O Ingress Controller avalia as regras, geralmente priorizando as mais específicas.
- **Reescrita de URL (nginx.ingress.kubernetes.io/rewrite-target):** muitas vezes, a sua aplicação back-end espera receber requisições em um caminho diferente do que é exposto pelo Ingress. Por exemplo, você pode expor `minhaempresa.com/meuapp`, mas o backend espera apenas `/` (a raiz) para o seu conteúdo. A anotação `nginx.ingress.kubernetes.io/rewrite-target` resolve isso, instruindo o Nginx Ingress Controller a reescrever o path da URL antes de encaminhá-lo ao Service backend. Você pode usar grupos de captura de expressões regulares (`$1`, `$2`, etc.) para ter um controle granular sobre qual parte da URL original será mantida após a reescrita. O path no Ingress Resource deve usar expressões regulares se você for usar esses grupos de captura.

Exemplo: se o cliente acessa `http://meudominio.com/api/v1/users` e você tem uma regra de path `/api(/|$)(.*)` com `rewrite-target: /$2`, o `$2` capturaria `/v1/users` (tudo após `/api/`). Assim, o backend receberia a requisição para `/v1/users`, e não para `/api/v1/users`.

Segurança com TLS e cert-manager: Criptografia Automatizada

A segurança é inegociável na web moderna. O **TLS/HTTPS** não é mais um diferencial, mas uma necessidade absoluta. Ele garante:

Criptografia: impede que dados sensíveis sejam interceptados e lidos;

Integridade: assegura que os dados não foram alterados durante a transmissão;

Autenticação: verifica a identidade do servidor, prevenindo ataques de *phishing*;

SEO e Reputação: motores de busca priorizam sites HTTPS, e navegadores alertam sobre sites HTTP não seguros;

Conformidade: muitos padrões de segurança e regulamentações (LGPD, GDPR, PCI DSS) exigem seu uso.

O desafio em ambientes dinâmicos como Kubernetes é a **gestão manual de certificados**. Certificados possuem validade (geralmente 90 dias com Let'sEncrypt), e renová-los manualmente para dezenas ou centenas de aplicações é propenso a erros, caro e causa interrupções de serviço.

É aqui que o **cert-manager** brilha. Ele é um add-on para Kubernetes que automatiza todo o ciclo de vida dos certificados X.509 (SSL/TLS). Ele opera como um "operador" nativo do Kubernetes, estendendo a API com CustomResourceDefinitions (CRDs):

- **Issuer / Cluster Issuer:** define como e de qual Autoridade Certificadora (CA) os certificados serão emitidos.
- **Issuer:** escopo de namespace (emite certificados para recursos no mesmo namespace);
- **ClusterIssuer:** escopo de cluster (emite certificados para recursos em qualquer namespace). Ideal para CAs públicas como Let's Encrypt. Configura o método de desafio (HTTP-01, DNS-01) para provar a posse do domínio. O **HTTP-01** é o mais comum com Ingress, pois o cert-manager usa o Ingress Controller para servir um token de validação em uma URL específica.

- **Certificate:** declara *qual* certificado deve ser emitido. Ele especifica o nome do domínio, o Issuer/ClusterIssuer a ser usado e onde o certificado resultante e a chave privada devem ser armazenados (em um Secret do Kubernetes). O cert-manager monitora este recurso para renovação automática.
- **Order e Challenge:** recursos temporários internos que o cert-manager usa para gerenciar a comunicação com a CA e a validação do domínio.

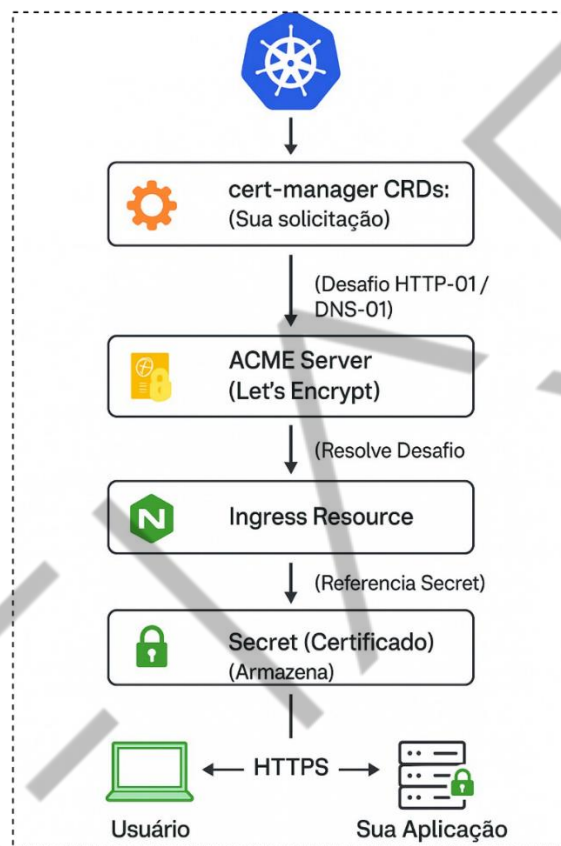


Figura 3 - Como funciona o cert-manager em alto nível
Fonte: Elaborado pelo autor (2025)

Integração com Ingress:

A integração é incrivelmente simples. No seu manifesto Ingress, você adiciona uma seção `tls` e uma anotação `cert-manager.io/cluster-issuer` (ou `issuer`). O cert-manager detecta essa anotação, cria o recurso `Certificate` internamente (ou você pode criá-lo separadamente), interage com a CA para obter o certificado e,

uma vez emitido, o armazena no Secret especificado. O NginxIngressController é então configurado para usar este Secret para o tráfego HTTPS.

```
# Exemplo de Ingress com TLS via cert-manager
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
name: minha-aplicacao-segura-ingress
annotations:
cert-manager.io/cluster-issuer: "letsencrypt-prod" # Nome do
seuClusterIssuer
spec:
ingressClassName: nginx
tls: # Habilita TLS para este Ingress
  - hosts:
    - meuapp.seunome.com # Domínio que será protegido
secretName: meuapp-tls-secret # Secret onde o cert-manager
armazenará o certificado
rules:
  - host: "meuapp.seunome.com"
    http:
      paths:
        - path: /
          pathType: Prefix
          backend:
            service:
              name: meu-backend-service
              port:
                number: 80
```

Código fonte 4: Exemplo de Ingress com TLS via cert-manager
Fonte: Elaborado pelo autor (2025)

Boas Práticas de Segurança em Kubernetes

Proteger suas aplicações vai muito além do HTTPS. O próprio cluster Kubernetes e a forma como você gerencia seus recursos exigem uma abordagem de segurança robusta. Aqui estão algumas boas práticas essenciais:

Princípio do Menor Privilégio (LeastPrivilege): conceda apenas as permissões mínimas necessárias para usuários, ServiceAccounts (identidades para Pods) e aplicações. Use o **RBAC (Role-Based Access Control)** para definir permissões granulares. Evite usar cluster-admin indiscriminadamente.

- **Roles vs. ClusterRoles:** roles definem permissões dentro de um namespace; ClusterRoles definem permissões em todo o cluster;
- **RoleBindings vs. ClusterRoleBindings:** ligam Roles ou ClusterRoles a usuários ou ServiceAccounts.

Network Policies: controle o tráfego de rede entre Pods (camada 4 do modelo OSI) e entre Pods e o exterior do cluster. Por padrão, todos os Pods em um namespace podem se comunicar. Network Policies permitem criar firewalls de rede baseados em labels, namespaces e IPs.

Segurança de Imagens e Registries:

- **Scanners de Vulnerabilidades:** utilize ferramentas como Trivy ou Clair para escanear suas imagens Docker em busca de vulnerabilidades conhecidas (CVEs) antes de implantá-las em produção. Integre isso no seu pipeline de CI/CD;
- **Imagens Mínimas:** prefira imagens base mínimas (alpine, distroless) para reduzir a superfície de ataque, pois menos software significa menos vetores de ataque;
- **Registries Privados e Seguros:** armazene suas imagens em registries privados e seguros, com autenticação forte e controle de acesso;
- **Gerenciamento de Secrets:** secrets do Kubernetes (para senhas, tokens, chaves API) são armazenados em Base64 por padrão, o que *não* é criptografia em repouso. Para produção, é fundamental usar soluções que criptografem os secrets de forma segura;
- **Sealed Secrets:** permite armazenar secrets criptografados no Git (seguro para controle de versão) e decifrá-los apenas no cluster.

- **HashiCorpVault / ExternalSecrets:** soluções robustas para gerenciamento centralizado de segredos, injetando-os nos Pods em tempo de execução;
- **Segurança em Tempo de Execução (Runtime Security):** monitore o comportamento dos containers e processos para detectar atividades anômalas ou maliciosas em tempo real;
- **Pod Security Admission (PSA):** um controle de admissão nativo do Kubernetes que aplica padrões de segurança a Pods. Por exemplo, pode impedir que um container rode como root ou acesse recursos sensíveis do host;
- **Ferramentas como Falco:** monitoram chamadas de sistema e eventos do kernel para detectar comportamentos suspeitos (ex: um processo inesperado acessando /etc/shadow).

Logs e Monitoramento:

- **Centralização de Logs:** agregue logs de todos os Pods e componentes do cluster em um sistema centralizado (ELK Stack, Grafana Loki etc.) para visibilidade, auditoria e solução de problemas;
- **Monitoramento de Segurança:** monitore os *audit logs* da API do Kubernetes para detectar acessos incomuns ou não autorizados ao cluster.

Manutenção e Atualizações: mantenha o Kubernetes e todos os seus componentes (NginxIngressController, cert-manager, etc.) sempre atualizados para aplicar patches de segurança. Monitore ativamente as Common Vulnerabilities and Exposures (CVEs) relacionadas às suas dependências.

CIS Benchmarks: siga os **CIS Benchmarks para Kubernetes e Docker**. São guias de configuração de segurança desenvolvidos por especialistas, que fornecem recomendações detalhadas para endurecer (harden) seus clusters e containers.

MERCADO, CASES E TENDÊNCIAS

Kubernetes in Action – Autor Marko Luksa - Editora Manning Publications

Este livro é um divisor de águas para qualquer profissional que deseja dominar o Kubernetes em profundidade. Escrito de forma acessível, mas tecnicamente robusta, ele explora cada conceito do Kubernetes – incluindo Deployment, Services e os fundamentos de networking que viabilizam o Ingress – com exemplos práticos e explicações claras. É uma leitura essencial para compreender a espinha dorsal das plataformas modernas de orquestração, capacitando você a entender não apenas "como", mas "por que" as soluções de gerenciamento de tráfego se encaixam no cenário de microsserviços e nuvem, e por que o Kubernetes se tornou a ferramenta dominante no mercado.

Cloud Native DevOps with Kubernetes: Building, Deploying, and Scaling Modern Applications in the Cloud – autores John Arundel e Justin Domingus. Editora O'Reilly Media

Focando na interseção crucial entre DevOps e Kubernetes, esta obra é um guia prático para construir e operar aplicações de forma eficiente em um ambiente nativo da nuvem. Ele detalha as melhores práticas de CI/CD, monitoramento, logging e, claro, gerenciamento de tráfego, tudo contextualizado para o ecossistema Kubernetes. Para quem busca entender como as empresas de ponta estão aplicando o Kubernetes para inovar e escalar suas operações de forma confiável – especialmente no que tange a gestão de acesso e balanceamento de carga para aplicações críticas – este livro oferece insights valiosos e casos de uso aplicáveis ao dia a dia do mercado.

Kubernetes Up and Running: Dive into the Future of Infrastructure – autores Brendan Burns, Joe Beda e Kelsey Hightower – Editora O'Reilly Media

Escrito por três dos arquitetos e engenheiros originais do Kubernetes no Google, este livro oferece uma perspectiva única e fundamental sobre a filosofia de design e a visão por trás da plataforma. Embora abranja os aspectos técnicos, seu grande valor reside em explicar *por quê* o Kubernetes foi criado e como ele revolucionou a forma como pensamos a infraestrutura. Para compreender as

tendências futuras do mercado, os desafios que as organizações enfrentam na adoção da nuvem e as direções que o gerenciamento de tráfego em ambientes distribuídos está tomando, mergulhar na mente dos criadores é um passo inestimável. Uma leitura obrigatória para quem deseja ir além do "como" e desvendar o "futuro" do setor.

EMANIP

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você embarcou em uma jornada completa pelo gerenciamento de tráfego no Kubernetes. Partindo da revisão de Containers e da arquitetura do Kubernetes, você compreendeu o papel fundamental do Ingress como a porta de entrada inteligente para suas aplicações. Aprofundou-se na configuração do NginxIngressController, dominando técnicas de roteamento por host e path, e explorou a poderosa reescrita de URL.

Finalmente, desvendou a automação do TLS com cert-manager, garantindo que suas aplicações estejam sempre seguras com HTTPS, e absorveu as boas práticas de segurança essenciais para qualquer ambiente Kubernetes em produção.

REFERÊNCIAS

CENTER FOR INTERNET SECURITY. **CIS benchmarks for Kubernetes**. Disponível em: <https://www.cisecurity.org>. Acesso em: 01 set. 2025.

CERT-MANAGER. **Documentação do cert-manager**. Disponível em: <https://cert-manager.io>. Acesso em: 01 set. 2025.

DOCKER. **Documentação oficial do Docker**. Disponível em: <https://docs.docker.com>. Acesso em: 01 set. 2025.

HELM. **Documentação oficial do Helm**. Disponível em: <https://helm.sh>. Acesso em: 01 set. 2025.

KUBERNETES. **Conceitos do Kubernetes**. Disponível em: <https://kubernetes.io/pt-br/docs/concepts/overview/components/>. Acesso em: 01 set. 2025.

KUBERNETES. **Ingress no Kubernetes**. Disponível em: <https://kubernetes.io/docs/concepts/services-networking/ingress/>. Acesso em: 01 set. 2025.

LET'S ENCRYPT. **Documentação oficial do Let's Encrypt**. Disponível em: <https://letsencrypt.org/docs/>. Acesso em: 01 set. 2025.

LUKSA, M. **Kubernetes in action**. Shelter Island: Manning Publications, 2021.

MINIKUBE. **Documentação oficial do Minikube**. Disponível em: <https://minikube.sigs.k8s.io/docs/>. Acesso em: 01 set. 2025.

NGINX INGRESS CONTROLLER. **Documentação oficial do Nginx Ingress Controller para Kubernetes**. Disponível em: <https://kubernetes.github.io/ingress-nginx/>. Acesso em: 01 set. 2025.

PALAVRAS-CHAVE

Kubernetes – Gerenciamento de Tráfego; IngressController; Certificados Digitais e TLS.

EMANIP



POSTECH